UDB VIRTUAL

Universidad Don Bosco, El Salvador

DOCENTE:

Alexander Alberto Siguenza

ACTIVIDAD:

Implementación de Interfaz Gráfica y Navegación con Jetpack Compose en el Sistema de Inventario SIRN

ALUMNOS:

Oscar Daniel Diaz Hernandez	DH252726
Henry Mauricio Peña Ramírez	PR251335
Manuel Alberto Mata Duran	MD130678
Krissia Lissette Eguizabal Fernandez	EF251345

ASIGNATURA:

Desarrollo de Aplicaciones para Dispositivos Móviles

FECHA DE ENTREGA:

18 de Octubre, 2026

GITHUB REPOSITORIO:
https://github.com/ManuelDuran95/Sistema_Inventario_UI

LINK DE VIDEO: https://youtu.be/kgzYeYycPzc

Implementación de Interfaz Gráfica y Navegación con Jetpack Compose en el Sistema de Inventario SIRN

Resumen

El presente informe técnico expone el desarrollo e implementación de la interfaz gráfica de usuario y la arquitectura de navegación para el Sistema de Inventario SIRN, migrando su lógica desde un entorno clásico basado en consola hacia una aplicación móvil nativa en la plataforma Android. Para lograr una experiencia moderna, mantenible y responsiva, se empleó Jetpack Compose como toolkit declarativo principal junto con los lineamientos de diseño de Material Design 3. La arquitectura interna del sistema aprovecha la separación de responsabilidades mediante el patrón ViewModel, el cual gestiona la reactividad del estado utilizando estructuras observables como mutableStateOf y mutableStateListOf. Asimismo, el flujo de pantallas y el control de credenciales se articularon mediante el componente NavHost y un NavController centralizado. Los resultados demuestran una reducción sustancial en la complejidad del código de UI, garantizando la persistencia temporal del estado durante reconfiguraciones de pantalla y facilitando una navegación fluida entre los módulos de autenticación y gestión de inventario.

Palabras clave: Jetpack Compose, UI Declarativa, Estado Reactivo, NavController, Material Design 3, Android.

Introducción

El desarrollo de aplicaciones móviles nativas en el ecosistema Android ha experimentado una transformación estructural a lo largo de la última década. Tradicionalmente, la construcción de interfaces de usuario dependía del paradigma imperativo, en el cual las vistas se definían en archivos XML independientes y eran manipuladas mediante llamadas explícitas desde el código Kotlin o Java. Este enfoque solía generar una fuerte acoplamiento entre la lógica de negocio y la capa de presentación, además de requerir un mantenimiento continuo para sincronizar manualmente el estado interno de la aplicación con la representación visual en pantalla.

Frente a estas limitaciones, el proyecto del Sistema de Inventario SIRN aborda la transición desde un prototipo funcional basado en consola de comandos hacia una aplicación móvil robusta y moderna. La necesidad de contar con una interfaz de usuario fluida, segura y escalable justificó la adopción de Jetpack Compose, el conjunto de herramientas moderno promovido por Google para la construcción nativa de UIs en Android.

El objetivo central de este trabajo es documentar la reestructuración del sistema SIRN, detallando tanto las bases teóricas del paradigma declarativo como la implementación práctica del paquete com.example.sistemainventario. A lo largo del informe se analiza cómo el patrón de diseño reactivo y la arquitectura basada en ViewModels permiten aislar la lógica de autenticación y la gestión de productos, garantizando una navegación coherente y una gestión eficiente del estado en la plataforma móvil.

Marco Teórico

1. Paradigma Declarativo frente al Enfoque Imperativo en Android

La diferencia fundamental entre el enfoque imperativo tradicional y el paradigma declarativo de Jetpack Compose radica en cómo se gestiona el ciclo de vida de los componentes visuales. En el modelo imperativo, el desarrollador instancia un árbol de vistas y emite órdenes explícitas para modificar las propiedades de cada elemento cuando ocurre un evento. Este procedimiento incrementa la probabilidad de inconsistencias, dado que el estado real de la aplicación puede desfasarse de los datos mostrados si alguna ruta de actualización es omitida.

Por el contrario, el paradigma declarativo describe cómo debe lucir la interfaz de usuario para un estado determinado. Cuando el estado cambia, el marco de trabajo ejecuta automáticamente las funciones compuestas relevantes mediante un proceso denominado recomposición. Jetpack Compose evalúa qué partes de la interfaz han sufrido alteraciones y recalcula únicamente los nodos necesarios del árbol de renderizado. Esto elimina la necesidad de manipular directamente las vistas, promoviendo un código más limpio, conciso y fácil de razonar.

2. Arquitectura Interna de Jetpack Compose

El funcionamiento de Jetpack Compose se sustenta en tres pilares arquitectónicos claramente delimitados dentro del ecosistema de bibliotecas de Android:

- **Compiler:** Es un complemento del compilador de Kotlin que analiza las funciones marcadas con la anotación @Composable. Su función principal es transformar las llamadas a funciones compuestas en instrucciones capaces de construir y actualizar la estructura de datos subyacente que representa la interfaz.
- **Runtime:** Constituye el núcleo de la gestión del estado y del árbol de composición. Se encarga de rastrear los cambios en los datos observables y programar los ciclos de recomposición cuando detecta mutaciones en el estado.
- **UI Layer:** Es la capa superior expuesta al desarrollador. Agrupa los componentes visuales fundamentales, el sistema de diseño, la gestión del diseño de páginas (layout), la detección de gestos de entrada y la integración con el lienzo de renderizado del dispositivo.

3. Componentes de Interfaz y Lineamientos de Material Design 3

Material Design 3 (M3) representa la evolución más reciente en los estándares de diseño visual de Google, priorizando la personalización, la accesibilidad y el dinamismo tipográfico y de color. Jetpack Compose integra M3 de forma nativa a través de componentes predefinidos que simplifican el cumplimiento de estos lineamientos.

Dentro del desarrollo del proyecto SIRN, se destacan contenedores como Scaffold, el cual proporciona la estructura estructural estándar recomendada por Android (incluyendo barras superiores, barras inferiores y contenedores de contenido con márgenes de seguridad predeterminados). Asimismo, el uso de superficies dinámicas como Card, campos de texto con validación como OutlinedTextField y botones de acción como Button aseguran una experiencia estandarizada y coherente con las guías de diseño móviles.

4. Gestión del Estado y Reactividad

La reactividad en Jetpack Compose depende de la capacidad del marco de trabajo para reaccionar ante cambios en los datos de entrada. Para lograr que un valor sea observable, se emplean primitivas de estado como mutableStateOf. Sin embargo, debido a que las funciones compuestas se vuelven a ejecutar frecuentemente durante la recomposición, los valores locales deben preservarse utilizando el bloque remember, el cual almacena el estado en la memoria del nodo de composición correspondiente.

Para mantener la coherencia de los datos ante eventos como la rotación de pantalla o la destrucción temporal de la actividad, la lógica del estado debe ser delegada a la clase ViewModel. Dentro de la arquitectura del proyecto, se emplea mutableStateListOf para manejar colecciones dinámicas de productos, permitiendo que la adición, eliminación o modificación de elementos en la lista desencadene automáticamente la actualización de los componentes compuestas asociados.

5. Sistema de Navegación y Control del Flujo

El control de la navegación en aplicaciones desarrolladas con Jetpack Compose abandona la transición tradicional entre múltiples Actividades o Fragmentos, adoptando una arquitectura de actividad única (Single Activity Architecture). El control del flujo de pantallas se articula a través de dos elementos centrales:

- **NavController:** Objeto central que gestiona el historial de navegación, la pila de pantallas (back stack) y el estado de la ruta activa.
- **NavHost:** Contenedor composable dentro del cual se define el mapa de navegación. Asocia cadenas de texto (rutas) con composables específicos, permitiendo realizar transiciones entre vistas de forma declarativa y segura.

Desarrollo e Implementación del Proyecto SIRN

La implementación del Sistema de Inventario SIRN dentro del paquete com.example.sistemainventario se diseñó siguiendo una arquitectura modular que separa la lógica de autenticación, la gestión de inventario y la interfaz de usuario. A continuación, se presenta la explicación detallada y los bloques de código fuente que conforman la solución.

1. Punto de Entrada: MainActivity.kt

La actividad principal configura el contenedor general de la aplicación. Aplica el tema visual de Material Design 3 y establece la estructura básica sobre la cual habrán de renderizarse las distintas pantallas.package com.example.sistemainventario

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
        MaterialTheme {
            Surface(modifier = Modifier.fillMaxSize()) {
                val navController = rememberNavController()
                AppNavigation(navController = navController)
            }
        }
    }
}
}
```

El componente `AppNavigation` define las rutas `"login"`, `"register"` y `"main"`. El uso de `popUpTo` dentro de la navegación asegura que, una vez que el usuario inicia sesión o se registra correctamente, las pantallas del flujo de autenticación sean removidas de la pila de navegación, impidiendo que el usuario regrese a ellas mediante el botón de retorno del sistema.

2. Gestión de Autenticación: AuthViewModel.kt

La lógica de control de usuarios y credenciales se aísla dentro de la clase `AuthViewModel`. Utiliza variables observables mediante `mutableStateOf` para exponer el estado actual del usuario autenticado y los mensajes de error.package com.example.sistemainventario

```
    var uiState by mutableStateOf(AuthState())
        private set
    fun onCredentialsChanged(user: String, pass: String) {
        uiState = uiState.copy(username = user, password = pass, error = null)
    }
    fun login(onSuccess: () -> Unit) {
        if (uiState.username == "admin" && uiState.password == "1234") {
            uiState = uiState.copy(isAuthenticated = true)
            onSuccess()
        } else {
            uiState = uiState.copy(error = "Credenciales inválidas")
        }
    }
}
```

3. Pantallas de Autenticación: AuthScreens.kt

La interfaz gráfica para los formularios de acceso y registro se construye en `AuthScreens.kt`. Estos compuestas gestionan su estado visual local (como el contenido de las cajas de texto) e interactúan directamente con el `AuthViewModel`.package com.example.sistemainventario

```
fun LoginScreen(viewModel: AuthViewModel, onLoginSuccess: () -> Unit) {
    val state = viewModel.uiState
```

```
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.Center) {
        Text("Sistema SIRN", style = MaterialTheme.typography.headlineMedium)
        OutlinedTextField(value = state.username, onValueChange = {
viewModel.onCredentialsChanged(it, state.password) }, label = { Text("Usuario") })
        OutlinedTextField(value = state.password, onValueChange = {
viewModel.onCredentialsChanged(state.username, it) }, label = { Text("Clave") })
        state.error?.let { Text(it, color = MaterialTheme.colorScheme.error) }
        Button(onClick = { viewModel.login(onLoginSuccess) }, modifier = Modifier.fillMaxWidth()) {
            Text("Ingresar")
        }
    }
}
```

4. Lógica de Gestión de Inventario: InventarioViewModel.kt

La estructura de datos de los productos y las operaciones de alta y baja del inventario están centralizadas dentro de `InventarioViewModel.kt`.package com.example.sistemainventario

```
class InventarioViewModel : ViewModel() {
    private val _items = mutableStateListOf<Producto>()
    val items: List<Producto> get() = _items
    fun agregarProducto(nombre: String, cantidad: Int) {
        if (nombre.isNotBlank() && cantidad > 0) {
            _items.add(Producto(id = _items.size + 1, nombre = nombre, cantidad = cantidad))
        }
```

```
    }
}
```

El uso de `mutableStateListOf` permite que cualquier adición o eliminación efectuada sobre `listaProductos` sea observada de manera inmediata por los componentes gráficos que consuman dicha variable, disparando la recomposición de los componentes correspondientes.

5. Interfaz Principal del Inventario: MainScreen.kt

La vista de administración del inventario integra un formulario de captura y una lista dinámica de productos renderizada mediante `LazyColumn`.package com.example.sistemainventario

```
fun AppNavigation(navController: NavHostController) {
    val authViewModel: AuthViewModel = viewModel()
    NavHost(navController = navController, startDestination = "login") {
        composable("login") {
            LoginScreen(viewModel = authViewModel) {
                navController.navigate("inventario") { popUpTo("login") { inclusive = true } }
            }
        }
        composable("inventario") {
            InventarioScreen(onLogout = { navController.navigate("login") })
        }
    }
```

}

Conclusiones

La migración del Sistema de Inventario SIRN desde un entorno textual hacia una interfaz gráfica nativa desarrollada en Jetpack Compose representó una optimización notable en términos de mantenibilidad, cohesión y experiencia de usuario. La adopción del paradigma declarativo redujo sustancialmente el volumen de código requerido para la actualización de datos visuales, eliminando las inconsistencias de estado habituales en los esquemas basados en XML.

Por otro lado, la arquitectura implementada mediante la integración de `ViewModel`, `mutableStateListOf` y la herramienta `NavHost` demostró una separación sólida de responsabilidades. La lógica del negocio y la autenticación permanecen aisladas de la capa de presentación, garantizando la persistencia de los datos ante eventos del ciclo de vida del dispositivo y asegurando un control de flujo limpio y seguro dentro del aplicativo móvil.

El Sistema de Inventario SIRN opera bajo un modelo de arquitectura limpia que desacopla la autenticación, la navegación y la gestión de datos en memoria. El flujo inicia en el punto de entrada de la aplicación, donde se controla el acceso mediante un módulo de validación de credenciales que distingue entre usuarios registrados y nuevas altas. Una vez validada la identidad, el sistema otorga acceso a un panel principal estructurado como un menú de opciones interactivas, desde el cual el usuario puede desplazarse de forma segura hacia los distintos submódulos operativos. Finalmente, la lógica de negocio del inventario centraliza el almacenamiento temporal de los registros mediante listas observables, permitiendo que cualquier alta, consulta o baja de activos refleje cambios inmediatos en la interfaz gráfica sin comprometer la estabilidad de la sesión.

Referencias

- Android Developers. (2023). *Jetpack Compose documentation*. Google. https://developer.android.com/jetpack/compose
- Android Developers. (2023). *Navigation with Compose*. Google. https://developer.android.com/jetpack/compose/navigation
- Lete, A. (2022). *Developing Android Apps with Jetpack Compose*. Packt Publishing.

- Material Design. (2023). *Material Design 3 specifications*. Google. https://m3.material.io/
- Smyth, N. (2022). *Android Studio Compose Edition: Developing Android Apps with Jetpack Compose*. Payload Media.